KIRO TRACK WRITE-UP

How Kiro was used on MishiPass

Kiro was used two ways on this project: as a terminal agent executing bounded, single-purpose corrections against the D1 access layer and test configuration, and as a phased task runner across five scoped branches that closed out Beta 1.5 UI corrections, a photo gallery, and a run of alignment fixes. Both modes stopped short of Kiro's spec, steering, and hooks workflow.

SECTION 01 — BOUNDED TASK EXECUTION

Scoped corrections, not the full spec/steering/hooks workflow

The earliest documented use of Kiro on this project was a single bounded correction task on the D1 data-access layer. Kiro removed an incorrect runtime foreign-key PRAGMA helper (`enableForeignKeys()`), confirmed that D1's default foreign-key enforcement already covered the case, and fixed the worker's Vitest configuration — the `nodejs_compat` flag and the D1 migration binding for the test pool.[1]

A small, separately scoped documentation correction followed: tightened wording on PBKDF2 iteration claims and a stale migration filename in the D1 schema spec.[1]

With the project owner's one-time explicit authorization, Kiro also committed, pushed, opened PR #6 (`feature/d1-access-layer` into `dev`), squash-merged it, and applied branch protection to `main` — one approving review required, no force pushes, no deletions.[1]

SECTION 02 — FIVE PHASED RUNS

Beta 1.5 gap closure, in five branches

The bulk of Kiro's usage was structured as multi-phase runs, each on its own branch, each producing a written run report. Kiro audited the codebase against the project's own Constitution and feature-status docs, applied fixes phase by phase, and flagged anything it could not complete along with the reason.

RUN · BRANCH	SCOPE	TESTS
Run 1 `fix/beta15-ui-corrections`	10 confirmed UI/UX bug fixes (nav, i18n, mode colors, sighting detail view, vet form sections, weight sync), plus a full photo-gallery feature: migration, repository, upload/list/delete/set-profile routes.	287→310
Run 2 `fix/beta15-alignment-corrections`	Persistent top nav on authenticated pages, global owner-language resolution, text-overflow fixes, cat registration promoted to its own tab, medical duplication removed from the cat detail page.	310→310
Run 3 `fix/beta15-alignment-round2`	Schema inventory before touching anything; full i18n coverage on the dashboard; stat-tile and mobile-tab sizing; dashboard return links added; birth-date/next-vaccine columns wired end to end.	310→313
Run 4 `fix/beta15-alignment-round3`	Definition-of-done audit against five specific claims; edit-cat API route (ownership-scoped); gallery visibility enforcement with a public/private serve split; creation success banner.	313→318
Run 5 `fix/beta15-run-a`	Cat-detail page audit (found the edit and gallery UI still missing); background image moved to Workers Static Assets; toast-style creation banner; Carbon icon swap; breeds show-more fold; Dependabot auto-merge workflow.	318→318

Across the five runs, the worker test suite grew from 287 to 318 passing tests, plus a steady 43 in the shared-validation workspace.[2] Each run report also lists what it deliberately deferred — the gallery UI on the cat detail page, an edit-cat form, and an app-level background image among them — rather than silently skipping it.

SECTION 03 — WORKING PATTERN

Audit first, fix in phases, name what's left

The recurring shape across all five runs was: audit the current state against the project's own docs (feature-status table, schema, Constitution) before changing anything; apply fixes as separately committed phases, one concern per commit; run the full test suite and `tsc --noEmit` at the end of each run; and record, per phase, exactly what was fixed, what file changed, and what was deferred with a stated reason.

Git and GitHub actions — commits, pushes, PRs, merges, branch protection — were gated behind explicit, one-time authorization from the project owner rather than run autonomously.[1] A commit-attribution error from one run (two commits misattributed due to per-commit rather than per-hunk batching) was caught and logged rather than silently left in the history.[4]

SECTION 04 — WHAT THIS WASN'T

No .kiro folder, no specs/steering/hooks

This was all bounded, single-purpose task execution rather than Kiro's IDE-native spec, steering, and hooks workflow.[3] The project's decision log records Kiro Track participation as paused at scaffold time because the optional `.kiro` folder was omitted; whether to opt back in, and under which framing, was left as an explicit open decision for the project owner rather than assumed.[3]

SOURCES

1. docs/kiro/kiro-track-checklist.md — bounded-task usage record (D1/FK correction, docs fix, authorized Git/GitHub actions).
2. Test counts compiled from docs/kiro/kiro-run-report.md, -alignment.md, -round2.md, -round3.md, -run-a.md.
3. docs/kiro/kiro-track-checklist.md — framing distinction between bounded-task usage and IDE-native spec/steering/hooks usage; open opt-in decision.
4. docs/kiro/kiro-run-report-run-a.md, Phase 6 — decision-log attribution correction.